
title: "Stereo TC MSCKF VIO — 0부터 100까지 완전 학습 가이드" date: 2026-05-27 type: insight
tags: [tc-msckf, stereo, opencv, complete-guide, learning, vio] related:

- "[[20260520_vio_lvio_msckf_thinking_cheatsheet]]"
 - "[[20260522_tc_msckf_overview]]"
 - "[[20260522_tc_msckf_port_plan]]"
 - "[[20260522_tc_msckf_port_journal]]"
 - "[[20260514_lc_ekf_cam_imu_fusion]]" status: reviewed
-

Stereo TC MSCKF VIO — 0부터 100까지 완전 학습 가이드

작성일 2026-05-27 · 작성 동기 cycle 1~5 를 거쳐 stereo TC MSCKF VIO 가 KITTI 0117 에서 작동 (full ATE 256 cm). 본인이 이 코드를 기반으로 학습하여 체득 할 수 있도록, 수식·개념·코드·이슈·튜닝까지 한 문서에 정리. **읽는 순서 (제안)** Part 1 (개념) → Part 3 (코드) → Part 4 (수식) → Part 5 (디버그) → Part 7 (튜닝). 처음에는 Part 1 만 읽어도 충분히 코드를 따라갈 수 있음. **대상 코드** D:\02_research\04_cpp_seg_msckf_vio @ tag v0.4.0-stereo-msckf (commit 7fc9906).

목차

- **Part 0** — 개요와 이 문서 사용법
 - **Part 1** — 개념 기초 (좌표계, 쿼터니언, EKF, IMU, 스테레오, MSCKF)
 - **Part 2** — 시스템 아키텍처 (LC vs TC, 파일 레이아웃, 데이터 흐름)
 - **Part 3** — 코드 워크스루 (어댑터 본체 정밀 해부)
 - **Part 4** — 수식 유도 (전파, 측정 자코비안, 널스페이스, 카이²)
 - **Part 5** — 디버그 사이클 1~5 의 기록과 교훈
 - **Part 6** — 결과와 LC 갭 분석
 - **Part 7** — 튜닝 가이드 (어떤 파라미터, 어떻게, 왜 영향)
 - **Part 8** — 방법론 회고
 - **Part 9** — Quick reference
-

Part 0 — 개요

0.1 본 프로젝트가 뭔가

KITTI/EuRoC 같은 **stereo + IMU** 데이터에서 차량/드론의 6-DOF 자세와 위치를 실시간으로 추정하는 VIO (Visual-Inertial Odometry) 시스템이다. 두 가지 backend 를 가진다:

Backend	풀네임	위치	ATE on KITTI 0117
LC EKF	Loosely-Coupled EKF	src/ekf/lc_ekf.cpp	152.96 cm (full 660f)
TC MSCKF	Tightly-Coupled MSCKF	src/msckf/, src/msckf_pipeline/	22.33 cm (50f), 256.26 cm

본 가이드는 **TC MSCKF (stereo)** 의 모든 면을 다룬다.

0.2 0~100 의 의미

- 0 = "VIO 가 뭔지도 모름"
- 30 = EKF / quaternion / 픽셀 자코비안 등 개별 개념 은 알지만 조립은 못 함
- 60 = OpenVINS 같은 코드 읽기 는 됨. 직접 짜라면 막힘
- 90 = stereo MSCKF 를 맨손으로 짜고 튜닝까지 가능
- 100 = 본인 데이터셋·로봇에 맞춤 으로 변형 가능 + 한계 진단

본 가이드는 30 → 90 사이의 격차를 채우는 데 집중. 0~30 의 기초는 [[20260520_vio_lvio_msckf_thinking_cheatsheet]] 참조.

0.3 학습 권장 순서

1. 첫 30분: Part 1 (개념) 만. 코드 안 보고.
2. 다음 1시간: Part 3 (코드 워크스루) 와 Part 1 을 왔다갔다 하며. 모르는 용어 나오면 Part 1 으로 돌아가기.
3. 반나절: Part 4 (수식) 를 펜으로 손유도. 종이에 직접.
4. 하루: Part 5 (디버그) 의 가설별 흐름 추적. 왜 그 가설을 세웠고 왜 틀렸는지/맞았는지.
5. 응용: Part 7 (튜닝) 의 파라미터 하나 골라서 실제로 바꿔보고 ATE 측정.

Part 1 — 개념 기초

1.1 좌표계와 표기

1.1.1 본 시스템의 좌표계 5종

약자	풀네임	정의
W	World (Global)	첫 IMU 의 정지 init 후 자세. Z 축 = up (gravity 반대).
I (= B)	IMU / Body	IMU 센서 frame. KITTI 의 oxts frame.
C0	Camera 0 raw	KITTI 의 image_00 (rectified 전).
RC0	Rectified Camera 0	cv::stereoRectify 후 cam0. epipolar 정렬됨.
RC1	Rectified Camera 1	같은 K, baseline 만큼 x 이동.

본 코드에서 MSCKF backend 는 **RC0, RC1** 만 사용. raw C0, C1 은 frontend (StereoTracker) 에서만 다룬다.

1.1.2 변환 표기

$T_{a \leftarrow b}$ = "b frame 의 점을 a frame 좌표로 옮기는 4×4 동차변환".

$$p_a = T_{a \leftarrow b} \cdot p_b, \quad T_{a \leftarrow b} = \begin{bmatrix} R_{a \leftarrow b} & t_{a \leftarrow b} \\ 0 & 1 \end{bmatrix}$$

- $R_{a \leftarrow b}$ = b 의 방향벡터를 a 좌표로.
- $t_{a \leftarrow b}$ = b 의 원점 의 a 좌표.

본 코드의 변수명 매핑:

변수	의미
T_cam0_imu	점을 imu → cam0 으로 보냄 (cam0 from imu)
T_rectcam0_imu	점을 imu → rectified cam0 으로
T_rectcam1_imu	점을 imu → rectified cam1 으로
R0_wi, init.R0	world ← imu (LC init 결과)
R_GtoI	global → imu (= R_wi.transpose(), JPL 표기)

1.1.3 한 가지 주의 — Global = World

OpenVINS 코드의 G 는 본 코드의 W 와 동일하다. 두 단어 같이 쓰임. gravity-aligned, IMU 첫 자세 기준 의 frame.

1.2 쿼터니언: Hamilton vs JPL

1.2.1 두 컨벤션의 차이

같은 회전을 표현하지만 부호 규칙 이 다르다.

측면	Hamilton (Eigen 기본)	JPL (OpenVINS, 다수 항공/우주)
곱셈 의미	$q_1 \otimes q_2 \rightarrow q_2$ 먼저 적용 후 q_1 (active, $p' = q_1 q_2 p (q_1 q_2)^*$)	$q_1 \otimes q_2 \rightarrow$ frame chain ($R_{GI} = R_{GC} I$)
회전 적용	$p' = q p q^{-1}$ (active)	$p_{frame2} = q^{-1} p_{frame1} q$ (passive)
q_w 위치	(w, x, y, z)	(x, y, z, w)
$R(q)$ 의 의미	active rotation (vector 회전)	$R_{G \rightarrow I}$ (passive, frame 변환)

한 줄 정리: Hamilton 의 $\otimes$ 와 R 곱은 반대 순서 (composite 가 오른쪽부터). JPL 의 $\otimes$ 와 R 곱은 같은 순서 (composite 가 왼쪽부터). 본 코드의 OV state 가 JPL → 모든 frame chain 이 왼쪽부터.

1.2.2 본 코드의 seam

본 코드는 boundary 에서 Hamilton, internal OpenVINS state 에서 JPL.

```
// 입력 boundary (LC 가 제공):
const Eigen::Matrix3d R0_wi;      // Hamilton matrix, world ← imu

// JPL 로 변환:
const Eigen::Matrix3d R_GtoI = R0_wi.transpose(); // global → imu
const Eigen::Vector4d q_GtoI = ov_core::rot_2_quat(R_GtoI); // JPL (x,y,z,w)

// 출력 boundary (latest_pose 반환):
out.R = state_-_imu->Rot().transpose(); // R_wi (Hamilton)
```

핵심: rot_2_quat 는 행렬을 JPL 컨벤션의 q 로 변환한다. JPL 의 $R_{G \rightarrow I}$ 의 q.

1.2.3 rot_2_quat: 행렬 → JPL 쿼터니언

OV 의 rot_2_quat(R) 의 핵심 식 (Shepperd's method):

1. $T = \text{tr}(R) = R_{11} + R_{22} + R_{33}$
2. 가장 큰 항을 찾아 안정성 보장:
 - 만약 $T > R_{ii}$ for all i : $q_w = \frac{1}{2}\sqrt{1+T}$, 나머지는 $\frac{1}{4q_w}(R_{32} - R_{23}, R_{13} - R_{31}, R_{21} - R_{12})$
 - 아니면 $q_i = \frac{1}{2}\sqrt{1+2R_{ii}-T}$, 나머지 비례
3. JPL convention 으로 부호 정규화: $q_w \geq 0$.

이 식이 수치적으로 안정 (sqrt 안의 값이 항상 ≥ 0). cycle 1 G2 에서 검증된 round-trip ($|R - R(q(R))| \approx 10^{-16}$) 이 이 식의 정확성 증거.

1.2.4 검증된 round-trip (cycle 1 G2)

| R_state.Rot() - R_GtoI | $\approx 1.13\text{e-}16$

→ 이 round-trip 은 정확. cycle 1 의 G2 가설은 ✓. quaternion 변환 자체에는 버그 없음.

1.3 EKF 기초

1.3.1 표준 EKF 한 사이클

상태 x , 공분산 P .

Propagate (시간 $t_{k-1} \rightarrow t_k$):

$$\mathbf{x}_k^- = f(\mathbf{x}_{k-1}^+, \mathbf{u}_k), \quad P_k^- = \Phi_k P_{k-1}^+ \Phi_k^T + Q_k$$

여기서 $\Phi_k = \partial f / \partial \mathbf{x}$, Q_k = process noise.

Update (측정 z_k):

$$\begin{aligned} \mathbf{r} &= \mathbf{z}_k - h(\mathbf{x}_k^-), \quad H = \partial h / \partial \mathbf{x}, \quad S = H P_k^- H^T + R \\ K &= P_k^- H^T S^{-1}, \quad \mathbf{x}_k^+ = \mathbf{x}_k^- \oplus K \mathbf{r}, \quad P_k^+ = (I - KH) P_k^- \end{aligned}$$

1.3.2 Error-state EKF (왜 필요한가)

쿼터니언은 unit-length 제약이 있어 공분산 전파 가 직접 안 된다. 해결: 상태를 명목값 $\bar{x}$ 와 작은 오차 δx 로 분리.

- 명목값은 만 비선형으로 적분
- 오차 (예: $\delta\theta$, 3-DOF) 만 공분산으로 추적
- 업데이트 시 δx 를 명목값에 합쳐서 초기화

OpenVINS, MSCKF, 거의 모든 modern VIO 가 이 방식.

State 차원 정리 (본 코드):

표현	차원	구성
Nominal IMU	16	$q(4) + p(3) + v(3) + b_g(3) + b_a(3)$
Error IMU	15	$\delta\theta(3) + \delta p(3) + \delta v(3) + \delta b_g(3) + \delta b_a(3)$
Nominal clone (pose)	7	$q(4) + p(3)$
Error clone	6	$\delta\theta(3) + \delta p(3)$

공분산 행렬 P 의 차원은 error state 기준. IMU + N clones 면 $(15 + 6N) \times (15 + 6N)$.

1.3.3 First Estimates Jacobian (FEJ)

왜 일관성 깨지는가: VIO 의 unobservable direction (yaw 절대값, 절대 position 4-DOF) 은 측정값으로 결정 불가. 그런데 EKF 의 자코비안 H 를 매 step 의 latest estimate 에서 다시 계산하면, 이 unobservable 방향에도 spurious information 이 누적됨 $\rightarrow P$ 가 false 하게 작아짐 $\rightarrow$ over-confident.

FEJ 의 해결: 자코비안을 최초 추정값 (= first estimate, FE) 에서 고정. 즉 propagation Jacobian Φ 와 measurement Jacobian H 모두 첫 linearization point 에서 평가.

수식: 표준 EKF 는 $H_k = \partial h / \partial x|_{x=\hat{x}_k}$. FEJ 는 $H_k = \partial h / \partial x|_{x=\hat{x}^{FE}}$ where $\hat{x}^{FE}$ = state 가 clone 으로 stored 될 때의 추정값.

본 코드: `opts.do_fej = true`. OV 의 표준. `state_->imu->set_fej(imu0)` 가 첫 추정값을 저장.

구체적 영향:

- yaw drift 의 축적 속도 감소 (yaw 가 정확한 unobservable 인 4-DOF 의 일부)
- position drift 도 비슷
- KITTI 같은 long-horizon (수백 m) 에서 차이 큼

해석: 측정 자코비안의 값 이 정확함보다 전체 EKF 의 consistency (state cov vs 실제 error 일치) 가 더 중요.

1.4 IMU 운동학

1.4.1 IMU 가 측정하는 것

- 자이로 w_m = 실제 각속도 + bias + noise
- 가속도 a_m = 실제 가속도 (gravity 포함) + bias + noise

$$w_m = w_{IB} + b_g + n_g, \quad a_m = a_{IB} - R_{G \rightarrow I} g_G + b_a + n_a$$

여기서 $g_G = (0, 0, +9.81)$ (본 코드의 G 가 up 이므로 + 부호).

1.4.2 연속시간 운동학

$$\dot{q}_{GI} = \frac{1}{2} \Omega(w_{IB}) q_{GI}, \quad \dot{p} = v, \quad \dot{v} = R_{I \rightarrow G}(a_m - b_a) + g_G$$

1.4.3 이산 시간 (OpenVINS 의 RK4)

본 코드는 OV 의 Propagator::propagate_and_clone 이 RK4 로 적분. 자세한 식은 Part 4.

1.4.4 Bias 모델

$$\dot{b}_g = n_{wg}, \quad \dot{b}_a = n_{wa}$$

random walk. NoiseManager::sigma_wb, sigma_ab 가 이 noise.

1.4.5 수식 표기 약속 (Ω , skew)

본 가이드와 OV 코드 전반에서 사용:

Skew-symmetric 행렬 $[v]_{\times}$ (cross product 의 행렬 표현):

$$[v]_{\times} = \begin{bmatrix} 0 & -v_z & v_y \\ v_z & 0 & -v_x \\ -v_y & v_x & 0 \end{bmatrix}, \quad [v]_{\times} u = v \times u$$

유용한 항등식 (자코비안 유도에서 쓰임): $R[v]_{\times} R^T = [Rv]_{\times}$.

JPL 의 $\Omega(w)$ 행렬 (쿼터니언 미분):

$$\Omega(w) = \begin{bmatrix} -[w]_{\times} & w \\ -w^T & 0 \end{bmatrix} \in \mathbb{R}^{4 \times 4}$$

이걸로 $\dot{q}_{GI} = \frac{1}{2}\Omega(w)q_{GI}$ 가 정의됨. ($w = w_{IB}$ = body 각속도)

Hamilton 컨벤션에서는 부호와 element 위치가 다른 Ω^H 사용. 본 코드는 OV 내부 JPL.

1.5 Stereo 기하

1.5.1 핀홀 모델 + 왜곡 (CamRadtan)

기본 핀홀:

$$u = f_x \cdot X/Z + c_x, \quad v = f_y \cdot Y/Z + c_y$$

여기서 (X, Y, Z) = 카메라 좌표계의 점.

CamRadtan (Radial-Tangential distortion 모델) 의 8-dim parameter:

$$\text{intr} = [f_x, f_y, c_x, c_y, k_1, k_2, p_1, p_2]$$

- k_1, k_2 : radial 왜곡 계수 (lens 의 fisheye/barrel 효과)
- p_1, p_2 : tangential 왜곡 (lens 미세 기울어짐)

왜곡 적용 (정규화 좌표 $(x_n, y_n) = (X/Z, Y/Z)$ 부터):

$$\begin{aligned} r^2 &= x_n^2 + y_n^2 \\ x_d &= x_n(1 + k_1 r^2 + k_2 r^4) + 2p_1 x_n y_n + p_2(r^2 + 2x_n^2) \\ y_d &= y_n(1 + k_1 r^2 + k_2 r^4) + p_1(r^2 + 2y_n^2) + 2p_2 x_n y_n \\ u &= f_x x_d + c_x, \quad v = f_y y_d + c_y \end{aligned}$$

본 코드는 rectified 스트림 사용 $\rightarrow$ distortion 항 모두 0 설정 ($\text{intr} \ll f_x, f_y, c_x, c_y, 0, 0, 0, 0$). CamRadtan 의 distort/undistort 함수가 항등 함수 로 동작.

대안 모델: **CamEqui** (Equidistant / fisheye, OV 의 다른 cam class). 본 코드 미사용.

1.5.2 Rectification

`cv::stereoRectify` 는 두 카메라를 공통 평면 위에 정렬:

- 두 cam 의 K 가 같아진다 ($f_x_{rect} = f_y_{rect}$, 공통 c_x, c_y)
- epipolar line = 수평 scanline $\rightarrow$ stereo matching 이 1D 검색
- baseline = 두 cam 의 x 좌표 차이

본 코드: `tracker.rectified_baseline()` 가 이 값을 반환.

1.5.3 Disparity $\rightarrow$ Depth

$$Z = \frac{f_x \cdot b}{u_L - u_R}$$

KITTI: $f_x \approx 721$, $b \approx 0.537$ m $\rightarrow$ 3m 깊이면 disparity ≈ 129 px.

1.5.4 $T_{rectcam1_rectcam0}$

Stereo rectified frame 에서 cam1 의 원점은 cam0 의 (+baseline, 0, 0) 에 있다. 따라서:

$$T_{rc1 \leftarrow rc0} = \begin{bmatrix} I & (-b, 0, 0)^T \\ 0 & 1 \end{bmatrix}$$

(cam1 frame 좌표는 cam0 frame 좌표보다 x 가 -b 만큼 작다. cam1 이 cam0 의 오른쪽 +b 에 있으므로 cam1 의 입장에서 cam0 frame 의 원점은 자기 왼쪽 -b.)

본 코드 (`apps/run_vio.cpp`):

```
Eigen::Matrix4d T_rectcam1_rectcam0 = Eigen::Matrix4d::Identity();
T_rectcam1_rectcam0(0, 3) = -tracker.rectified_baseline();
Eigen::Matrix4d T_rectcam1_imu = T_rectcam1_rectcam0 * T_rectcam0_imu;
```

검증 (KITTI 0117): $T_{rectcam0_imu}(0,3) \approx -0.314$, baseline 0.537 $\rightarrow$
 $T_{rectcam1_imu}(0,3) \approx -0.851$. ✓

1.6 MSCKF — Multi-State Constraint KF 의 본질

1.6.1 왜 "multi-state constraint"

한 feature 가 N 개 frame 에서 보인다면, N 개 camera pose 사이에 기하학적 제약 이 있다 (모두 같은 3D 점을 본다는 사실).

LC EKF 는 이 제약을 VO 가 미리 풀어 (예: PnP RANSAC) pose 만 EKF 에 전달.

TC MSCKF 는 이 제약을 EKF 가 직접 사용. feature 의 3D 위치를 임시로 추정 후 제약을 manifold 에서 marginalize (null-space projection).

1.6.2 Sliding window of clones

state = [

```

IMU state (16-dim)      ← 현재 시점
clone 0 (7-dim pose)    ← t-N 시점의 IMU pose
clone 1 (7-dim pose)
...
clone N-1 (7-dim pose) ← t-1 시점
]

```

본 코드: `opts.max_clone_size = 30` (cycle 5 final). $30 \text{ frame} \times 0.1\text{s} = 3\text{초 window}$.

1.6.3 MSCKF update 6-step

UpdaterMSCKF::update 의 흐름:

1. **Clean**: feature 의 measurement 중 clone time 에 없는 것 제거. < 2 면 drop.
2. **Build clone poses**: 각 cam_id 의 각 clone 마다 $R_{G \rightarrow Ci}, p_{Ci,G}$ 계산.
3. **Triangulate**: feature 마다 3D 위치 추정 (Gauss-Newton refine).
4. **Per-feature**:
 - 측정 자코비안 H_f (feature), H_x (state) 계산
 - **Null-space projection**: H_f 의 null-space 에 H_x, r 사영 $\rightarrow$ feature 사라짐
 - **Chi² gate**: $\chi^2 = r^T (H_x P H_x^T + \sigma_{px}^2 I)^{-1} r$. > threshold 면 drop.
5. **Measurement compression QR**: 큰 H_x 를 QR 로 압축.
6. **EKF update**: 표준 EKF 식.

1.6.4 Null-space projection 의 핵심 아이디어

각 feature 가 m 개 측정값을 가지면 residual r 은 $2m$ 차원, H_f 는 $2m \times 3$. H_f 의 left null-space 는 $2m - 3$ 차원.

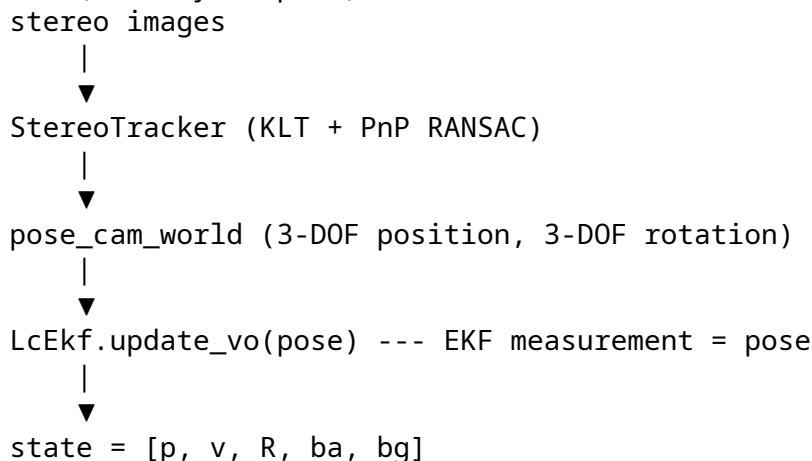
$$A^T H_f = 0 \implies A^T r = A^T H_x \delta x$$

$\rightarrow$ feature state 가 사라진 EKF 측정. 차원 $2m - 3$.

본 코드의 UpdaterHelper::nullspace_project_inplace 가 이걸 Givens rotation 으로 inplace 구현.

1.7 LC EKF vs TC MSCKF 정보 흐름

LC EKF (loosely-coupled):




```

TC MSCKF (tightly-coupled):
  stereo images
    |
    ▼
  StereoTracker (KLT, but feature *measurements* exposed)
    |
    ▼ tracked features {id, u_l, v_l, u_r, v_r, valid}
  MsckfPipeline.feed_camera(t, left, right)
    |
    ▼ Feature.uvs[cam_id][m] = (u, v)
  UpdaterMSCKF.update()
    |
    |-- build clones_cam (cam_id × clone × pose)
    |-- triangulate each feature
    |-- per-feature: H_f, H_x, null-space, chi2
    `-- compress + EKF update
    ▼
  state = [IMU 16-dim + N×7-dim clones]

```

핵심 차이: LC 는 feature 정보를 pose 로 압축 한 뒤 EKF 가 받는다. TC 는 feature 측정값 그 자체를 EKF 가 본다.

→ TC 가 원리상 더 많은 정보 사용 가능. 단 튜닝 부담 크다 (sigma_pix, chi2, FEJ, NoiseManager 등).

Part 2 — 시스템 아키텍처

2.1 파일 레이아웃

```

D:\02_research\04_cpp_seg_msckf_vio\
|--- apps/run_vio.cpp           ← 메인 엔트리. --engine=lc|msckf
|--- include/
|   |--- core/types.hpp        ← ImuData, CamData, GtData
|   |--- io/
|   |   |--- dataset_reader.hpp ← 인터페이스
|   |   |--- euroc_reader.hpp
|   |   `--- kitti_raw_reader.hpp
|   |--- ekf/                  ← LC EKF (검증된 baseline)
|   |   |--- lc_ekf.hpp
|   |   `--- imu_propagator.hpp
|   |--- frontend/
|   |   `--- stereo_tracker.hpp ← KLT + ORB + PnP + cycle 5 의 cam1 노출
|   |--- msckf/
|   |   |--- types/           ← IMU, PoseJPL, Landmark, Vec
|   |   |--- state/           ← State, StateOptions, Propagator, StateHelper
|   |   |--- update/          ← UpdaterMSCKF, UpdaterHelper, UpdaterOptions
|   |   |--- feat/            ← Feature, FeatureInitializer
|   |   |--- cam/             ← CamBase, CamRadtan, CamEqui
|   |   |--- utils/           ← quat_ops, NoiseManager, print, sensor_data
|   |   `--- init/InitializerHelper.hpp ← gram_schmidt (cycle 2)

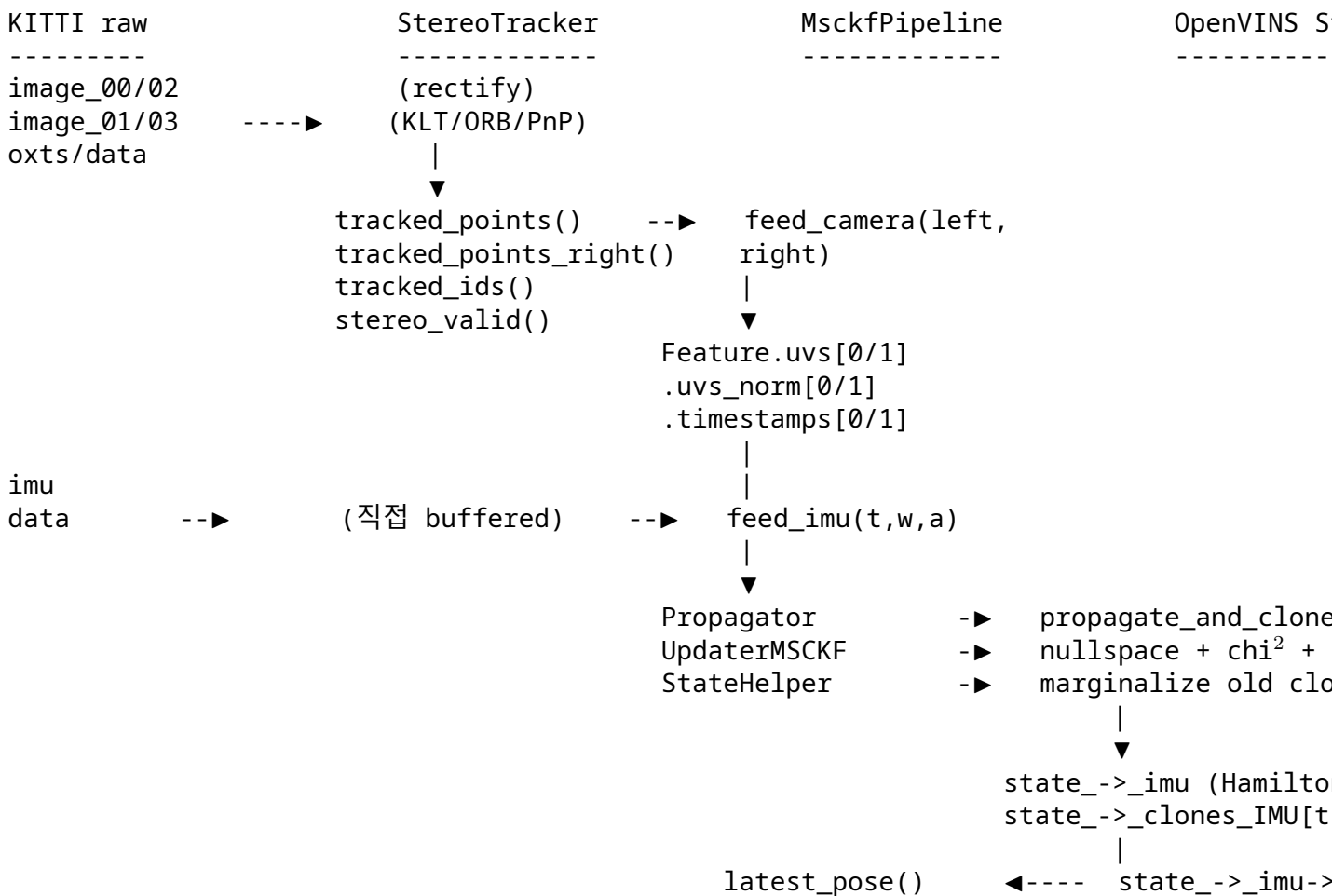
```

```

|   `--- msckf_pipeline/
|       `--- msckf_pipeline.hpp    ← VioManager 대체 어댑터
|--- src/
|   |--- ekf/, io/, frontend/      ← 위 헤더의 본체
|   |--- msckf/                    ← 0v 포트의 본체
|   `--- msckf_pipeline/           ← 어댑터 본체 (~300줄)
|--- tools/
|   `--- probe_kitti_imu_variance.cpp ← cycle 3 H1 진단
|--- config/                        ← yaml run config
|--- data/kitti_raw/...
|--- docs/
|   |--- insight/                  ← 본 문서 포함
|   `--- Practice/                ← 진행 추적
|--- CMakeLists.txt

```

2.2 데이터 흐름



2.3 Hamilton ↔ JPL seam 위치

코드에서 컨벤션이 바뀌는 단 4 지점:

1. **ctor 입력**: $R0_wi$ (Hamilton) → R_GtoI (JPL matrix) → q_GtoI (JPL quat, x-y-z-w)
2. **ctor 의 extrinsic**: T_cam_imu (Hamilton 4×4) → $[q_CtoI, p_IinC]$ (JPL 7-dim)
3. **feed_camera 의 픽셀**: 항상 Hamilton 정의의 pixel space — 컨벤션 차이 없음

4. **latest_pose** 출력: state->_imu->Rot() 는 JPL 의 R_Gtol $\rightarrow$.transpose() 로 Hamilton R_wi

다른 모든 내부 연산은 OpenVINS 의 JPL 그대로.

2.4 StereoTracker $\rightarrow$ MsckfPipeline 인터페이스

```
// 매 카메라 frame:
const auto& ids    = tracker.tracked_ids();           // size N
const auto& pts_l  = tracker.tracked_points();       // size N, cam0 픽셀
const auto& pts_r  = tracker.tracked_points_right(); // size N, cam1 픽셀 (cycle 5)
const auto& s_ok   = tracker.stereo_valid();         // size N, bool

std::vector<MsckfPipeline::TrackedFeat> left, right;
for (size_t i = 0; i < pts_l.size(); ++i) {
    left.push_back({ids[i], pts_l[i].x, pts_l[i].y});
    if (s_ok[i]) {
        right.push_back({ids[i], pts_r[i].x, pts_r[i].y});
    }
}
msckf->feed_camera(cam.timestamp, left, right);
```

중요한 invariant:

- left.size() == N (모든 alive track)
 - right.size() <= N (stereo 매칭 valid 한 subset)
 - right[i].id 는 반드시 left[j].id 와 매칭되는 ID 가 있어야 함 (아니면 backend drop)
-

Part 3 — 코드 워크스루

본 part 는 학습의 주력 부분. 코드를 왜 그렇게 짰는지 + 수식과의 매핑 까지.

3.1 MsckfPipeline ctor 정밀 해부

```
MsckfPipeline::MsckfPipeline(double t0,
                               const Eigen::Matrix3d& R0_wi,
                               const Eigen::Vector3d& mean_accel,
                               const Eigen::Vector3d& mean_gyro,
                               double gravity_mag,
                               const Eigen::Matrix4d& T_cam0_imu,
                               const Eigen::Matrix4d& T_cam1_imu,
                               double fx_rect, double fy_rect,
                               double cx_rect, double cy_rect,
                               int img_width, int img_height)
```

3.1.1 StateOptions

```
ov_msckf::StateOptions opts;
opts.do_fej = true;           // FEJ on: 일관성 보존
```

```

opts.do_calib_* = false; // 모든 calibration 끄: KITTI 는 미리 calib 됨
opts.max_clone_size = env_int("MSCKF_MAX_CLONES", 30); // 30 = 3초 window @10Hz
opts.max_slam_features = 0; // SLAM feature 안 씬, MSCKF only
opts.max_msckf_in_update = 1000; // 한 update 의 최대 feature 수
opts.num_cameras = 2; // cycle 5: stereo
opts.feats_rep_msckf = LandmarkRepresentation::Representation::GLOBAL_3D;

```

왜 GLOBAL_3D: feature 의 3D 위치를 global frame 에 표현. 대안은 ANCHORED_* (특정 cam frame 의 inverse depth). 본 코드는 anchor 식 안 씬.

3.1.2 IMU state 초기화

```

const Eigen::Matrix3d R_GtoI = R0_wi.transpose(); // Hamilton → JPL
const Eigen::Vector4d q_GtoI = ov_core::rot_2_quat(R_GtoI);
const Eigen::Vector3d bg = mean_gyro;
const Eigen::Vector3d ba = Eigen::Vector3d::Zero(); // cycle 3 H2: ba=0

// IMU 16-dim 상태 = [q(4), p(3), v(3), bg(3), ba(3)]
Eigen::Matrix<double, 16, 1> imu0;
imu0.block<4, 1>(0, 0) = q_GtoI;
imu0.block<3, 1>(4, 0) = Eigen::Vector3d::Zero(); // p = origin
imu0.block<3, 1>(7, 0) = Eigen::Vector3d::Zero(); // v = 0 (seed 는 나중)
imu0.block<3, 1>(10, 0) = bg;
imu0.block<3, 1>(13, 0) = ba;
state->_imu->set_value(imu0);
state->_imu->set_fej(imu0); // FEJ 의 첫 추정값

```

3.1.3 초기 공분산 (cycle 4 H5 fix)

```

Eigen::MatrixXd init_cov = std::pow(0.02, 2) * I_15;
init_cov.block<3,3>(0,0) = std::pow(0.02, 2) * I_3; // q (0.02 rad std)
init_cov.block<3,3>(3,3) = std::pow(0.05, 2) * I_3; // p (5 cm std)
init_cov.block<3,3>(6,6) = std::pow(0.01, 2) * I_3; // v (1 cm/s std)
// bg/ba: 기본 (0.02)2 유지
StateHelper::set_initial_covariance(state_, init_cov, {state->_imu});

```

왜 중요: OpenVINS 의 StaticInitializer 가 이 covariance 를 명시적으로 세팅한다. 본 어댑터는 그 함수를 호출 안 하므로 수동 으로 적용. cycle 3 H2 단계까지 이게 누락되어 너무 작은 init covariance 가 χ^2 lock-out 의 근본 메커니즘 중 하나.

3.1.4 카메라 calib 등록 (cycle 5 stereo)

```

auto set_calib = [&](std::size_t cam_id, const Eigen::Matrix4d& T_cam_imu) {
    Eigen::Matrix3d R_CI = T_cam_imu.block<3, 3>(0, 0);
    Eigen::Vector3d t_CI = T_cam_imu.block<3, 1>(0, 3);
    Eigen::Matrix<double, 7, 1> ext;
    ext.block<4, 1>(0, 0) = ov_core::rot_2_quat(R_CI); // q_CtoI (JPL)
    ext.block<3, 1>(4, 0) = t_CI; // p_IinC
    state->_calib_IMUtoCAM.at(cam_id)->set_value(ext);
    state->_calib_IMUtoCAM.at(cam_id)->set_fej(ext);
};

```

```

};
set_calib(0, T_cam0_imu);
set_calib(1, T_cam1_imu);

// 공통 K
Eigen::Matrix<double, 8, 1> intr;
intr << fx_rect, fy_rect, cx_rect, cy_rect, 0.0, 0.0, 0.0, 0.0;
for (std::size_t cam_id : {0u, 1u}) {
    state->_cam_intrinsics.at(cam_id)->set_value(intr);
    state->_cam_intrinsics.at(cam_id)->set_fej(intr);
    auto cam = std::make_shared<ov_core::CamRadtan>(img_width, img_height);
    cam->set_value(intr);
    state->_cam_intrinsics_cameras.insert({cam_id, cam});
}

```

JPL extrinsic 의 의미:

- q_{CtoI} = camera-from-imu 의 JPL quat. $\rightarrow$ Rot() 시 $R_{C \leftarrow I}$ 반환.
- p_{IinC} = imu origin 을 camera frame 으로 표현한 3-벡터.

이게 OV 의 _calib_IMUtoCAM 의 약속.

3.1.5 Propagator + Updater

```

ov_msckf::NoiseManager noises;
noises.sigma_a = 5.886e-3; // cycle 3 H3a: KAIST IMU 값
noises.sigma_a_2 = pow(noises.sigma_a, 2);
prop_ = std::make_unique<Propagator>(noises, gravity_mag);

upd_opts_ = std::make_unique<UpdaterOptions>();
upd_opts_->chi2_multipler = env_double("MSCKF_CHI2_MULT", 5.0);
upd_opts_->sigma_pix = env_double("MSCKF_SIGMA_PIX", 5.0);
upd_opts_->sigma_pix_sq = pow(upd_opts_->sigma_pix, 2);

```

$\rightarrow$ env 로 즉시 sweep 가능: MSCKF_SIGMA_PIX=2.0 ./run_vio ...

3.2 feed_imu

```

void MsckfPipeline::feed_imu(double t, const Vector3d& gyro, const Vector3d& accel) {
    ov_core::ImuData data;
    data.timestamp = t;
    data.wm = gyro;
    data.am = accel;
    prop_->feed_imu(data);
    last_imu_t_ = t;
    last_imu_w_ = gyro;
    last_imu_a_ = accel;
}

```

중요한 host-side 패턴 (apps/run_vio.cpp):

```

if (use_msckf) {

```

```

// 전체 IMU 스트림을 *t0 이전 샘플도* 포함해 미리 다 feed.
for (size_t k = 0; k < imu_data.size(); ++k) {
    msckf->feed_imu(imu_data[k].timestamp, imu_data[k].gyro, imu_data[k].accel);
}
}

```

왜: OV 의 Propagator::select_imu_readings 가 boundary 에서 주변 샘플 을 interpolate. 첫 cam 시점 직전 샘플도 필요. 또한 마지막 cam 시점 직후 샘플도. → 전체 미리 buffer. cycle 4 의 3개 boundary fix 중 upfront IMU feed 가 이 패턴.

3.3 feed_camera 정밀 해부

이게 본 코드의 심장. 5 step.

3.3.1 Step 1 — Propagate + clone

```

if (first_camera_) {
    first_camera_ = false;
    StateHelper::augment_clone(state_, Vector3d::Zero());
} else {
    prop_->propagate_and_clone(state_, t);
}

```

- 첫 frame: 시간 진행 없이 현재 IMU pose 를 clone 으로 추가.
- 두 번째 이후: IMU 적분으로 state 를 t 까지 전파 + clone 추가.

왜 첫 frame 특수 처리: propagate_and_clone 안의 select_imu_readings 는 $dt > 0$ 가정 → t_0 에서 다시 호출하면 abort. cycle 4 boundary fix.

3.3.2 Step 2 — Feature DB 업데이트 (stereo)

```

auto push_measurement = [&](Feature& feat, size_t cam_id, float u, float v) {
    feat.uvs[cam_id].push_back({u, v});
    feat.uvs_norm[cam_id].push_back({(u - cx_) / fx_, (v - cy_) / fy_});
    feat.timestamps[cam_id].push_back(t);
};

// 2a. cam0: 모든 alive track
for (const auto& tf : left) {
    seen_this_frame.insert(tf.id);
    auto& feat = fetch_or_create(tf.id);
    push_measurement(feat, 0, tf.u, tf.v);
}
// 2b. cam1: 이미 존재하는 feature 에만
for (const auto& tf : right) {
    auto it = feature_db_.find(tf.id);
    if (it == feature_db_.end()) continue; // right without left → drop
    push_measurement(it->second, 1, tf.u, tf.v);
}

```

왜 right without left → drop: feature의 temporal 연속성은 cam0 측정값으로 정의. cam1 only 측정은 triangulation baseline 으로부터 의미 있는데 그 시점만의 측정 — temporal information 0. backend 가 사용하기 어려움.

3.3.3 Step 3 — Lost feature 수집

```
for (auto& kv : feature_db_) {
    if (seen_this_frame.count(kv.first)) continue; // 살아있음
    std::size_t total_meas = 0;
    for (const auto& pair : kv.second->timestamps) total_meas += pair.second.size();
    if (total_meas < 2) {
        ids_to_erase.push_back(kv.first);
        continue;
    }
    feats_to_update.push_back(kv.second);
    ids_to_erase.push_back(kv.first);
}
```

개념: MSCKF 는 feature 가 시야에서 사라지는 순간 (또는 충분히 많은 frame 에서 관찰된 후) update 에 사용. 시야 안에 있는 feature 는 대기 — 매 frame 의 측정값 누적.

→ 이 패턴이 sliding window 와 결합해 “한 feature 가 N frame 의 pose 를 동시 제약” 의 정보를 끌어냄.

3.3.4 Step 4 — MSCKF update + 카운터

```
const int submitted = feats_to_update.size();
updater->update(state_, feats_to_update);
const int consumed = feats_to_update.size(); // in-place erase 됨

feats_submitted_total_ += submitted;
feats_consumed_total_ += consumed;
// ... log every 25 frames
```

중요한 관찰: feats_to_update 가 UpdaterMSCKF 안에서 in-place mutate:

- < 2 measurements → erase
- triangulation 실패 → erase
- χ^2 실패 → erase
- 통과한 것만 살아남음 (그러나 to_delete = true 마킹)

→ submitted - consumed = drop 수. consumed / submitted = accept rate.

3.3.5 Step 5 — Marginalize old clone (Schur complement 의 의미)

```
while (state->_clones_IMU.size() > opts.max_clone_size) {
    StateHelper::marginalize_old_clone(state_);
}
```

가장 오래된 clone 을 제거하면서 정보 손실 없이 차원만 줄임. 이게 **Schur complement** 의 의미: 상태를 $x = (x_a, x_b)$ 로 분리 (x_b = 제거할 clone, x_a = 나머지). 공분산:

$$P = \begin{bmatrix} P_{aa} & P_{ab} \\ P_{ba} & P_{bb} \end{bmatrix}$$

x_b 를 marginalize 한 후의 x_a 의 공분산:

$$P_{aa}^{\text{marg}} = P_{aa} - P_{ab}P_{bb}^{-1}P_{ba}$$

핵심: $P_{ab}P_{bb}^{-1}P_{ba}$ 항이 x_b 와 x_a 의 cross-correlation 정보를 P_{aa}^{marg} 로 흡수. → 단순 drop 이 아니라 정보 보존.

비교:

- 단순 drop: $P_{aa}^{\text{drop}} = P_{aa}$ — cross 정보 손실
- Schur: P_{aa}^{marg} — cross 정보 보존. 가장 오래된 clone 이 보던 모든 feature 의 기하 제약 이 나머지 state 의 covariance 로 누적.

이게 MSCKF 가 "sliding window 사이즈 제한" + "장기간 일관성" 두 가지를 모두 갖는 비결.

3.4 StereoTracker 의 cycle 5 확장

핵심 변경 (3 곳):

3.4.1 새 멤버 + accessor

```
std::vector<cv::Point2f> prev_pts_r_;    // size N
std::vector<bool> stereo_valid_;        // size N
```

```
const std::vector<cv::Point2f>& tracked_points_right() const { return prev_pts_r_; }
const std::vector<bool>& stereo_valid() const { return stereo_valid_; }
```

3.4.2 Bootstrap (첫 frame)

```
auto match = orb_stereo_match(gray_l, gray_r, {}, target_features_);
auto pts3d = stereo_triangulate(match.pts_l, match.pts_r);
for (size_t i = 0; i < pts3d.size(); ++i) {
    if (pts3d[i][2] > 0.1 && pts3d[i][2] < 50.0) {
        prev_pts_l_.push_back(match.pts_l[i]);
        prev_pts_r_.push_back(match.pts_r[i]);    // cycle 5
        stereo_valid_.push_back(true);            // cycle 5
        // ... map_pts_world_, prev_track_ids_
    }
}
```

3.4.3 후속 frame (process)

```
// step 1 temporal KLT + step 2 PnP 후 살아남은 tracked_pts 에 대해:
std::vector<cv::Point2f> tracked_pts_r;
std::vector<bool> tracked_valid;
if (!tracked_pts.empty()) {
    std::vector<uchar> stereo_ok;
```



```

    auto pts_r_curr = stereo_klt(gray_l, gray_r, tracked_pts, stereo_ok);
    for (size_t i = 0; i < tracked_pts.size(); ++i) {
        const bool ok = (i < stereo_ok.size()) && stereo_ok[i];
        tracked_pts_r.push_back(ok ? pts_r_curr[i] : cv::Point2f(-1.f, -1.f));
        tracked_valid.push_back(ok);
    }
}
// step 4 (new feature ORB stereo match) 에서도 right + valid 같이 push
// 마지막: prev_pts_r_ = tracked_pts_r, stereo_valid_ = tracked_valid

```

알려진 이슈 (Part 6 의 LC 갭 분석 참조):

- 후속 frame 의 right 픽셀은 KLT 결과 → sub-pixel drift 가능
- bootstrap/new feature 의 right 는 ORB descriptor match → 더 정확
- 시간이 갈수록 KLT 기반 cam1 측정값의 품질이 떨어질 가능성

Part 4 — 수식 유도

4.1 IMU 전파 (continuous → discrete)

4.1.1 연속시간

q_{GI} 의 update: $dq/dt = 0.5 * \Omega(w) * q$
 p 의 update: $dp/dt = v$
 v 의 update: $dv/dt = R_{IG} * (a_m - b_a) + g_G$ where $R_{IG} = R_{GI}^T$
 b_g, b_a : random walk

4.1.2 OV 의 RK4 (개요)

OV 의 Propagator::predict_and_compute 는 4-th order Runge-Kutta:

$$x(t + \Delta t) = x(t) + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

여기서 k_i 는 IMU 측정값을 mid-step 에서 interpolate 한 결과로 계산. select_imu_readings 가 이 interpolation 을 담당 — cam timestamp 좌우 IMU 샘플이 필요 한 이유.

4.1.3 Error-state 전파

오차 상태 $\delta x = [\delta \theta, \delta p, \delta v, \delta b_g, \delta b_a]$ 의 dynamics:

$$\dot{\delta x} = F \delta x + G n$$

F 의 핵심 블록 (지면 한계로 일부):

- $F_{\theta\theta} = -[w_{IB}]_{\times}$ (각속도 skew)
- $F_{vp} = I_3$
- $F_{v\theta} = -R_{I \rightarrow G}[a_m - b_a]_{\times}$
- $F_{vb_a} = -R_{I \rightarrow G}$

이산화: $\Phi_k = \exp(F\Delta t) \approx I + F\Delta t$ (1차) 또는 더 높은 차수.

공분산:

$$P_k = \Phi_k P_{k-1} \Phi_k^T + G_k Q G_k^T$$

본 코드의 Q 는 `NoiseManager::sigma_*` 로 구성.

4.2 측정 자코비안 (stereo feature)

4.2.1 측정 모델 (OV 의 실제 분해)

OV UpdaterHelper::get_feature_jacobian_full 의 정확한 분해:

$$\mathbf{p}_{F,I_i} = R_{I \leftarrow G}(\mathbf{p}_F - \mathbf{p}_{I,G}) \quad (\text{IMU clone frame 의 feature})$$

$$\mathbf{p}_{F,C_i} = R_{C \leftarrow I} \cdot \mathbf{p}_{F,I_i} + \mathbf{p}_{I,C} \quad (\text{camera frame 의 feature})$$

$$\mathbf{z} = \pi_{\text{dist}}(\mathbf{p}_{F,C_i})$$

기호:

- $\mathbf{p}_F$ = feature 의 global 3D 위치 (state 의 일부)
- $\mathbf{p}_{I_i,G}$ = i -번째 IMU clone 의 global 위치
- $R_{I \leftarrow G}$ = i -번째 IMU clone 의 회전 (JPL)
- $(R_{C \leftarrow I}, \mathbf{p}_{I,C})$ = camera-imu extrinsic (state 에 등록)
- $\pi_{\text{dist}}(\cdot)$ = 핀홀 + radtan distortion (본 코드는 rectified 라 distort 함등)

이 분해는 camera pose 가 state 에 직접 없고 IMU clone + extrinsic 으로 표현 됨을 명시.

4.2.2 Chain rule + 자코비안

$\mathbf{X}_C := \mathbf{p}_{F,C_i}$. 정규화 좌표 $(x_n, y_n) = (X_C/Z_C, Y_C/Z_C)$.

Pinhole projection 자코비안 ($Z = Z_C$):

$$\frac{\partial \mathbf{z}}{\partial \mathbf{X}_C} = \frac{1}{Z} \begin{bmatrix} f_x & 0 & -f_x X/Z \\ 0 & f_y & -f_y Y/Z \end{bmatrix}$$

State 의 각 변수에 대한 derivative (error state 기준):

- Feature global 위치: $\frac{\partial \mathbf{X}_C}{\partial \mathbf{p}_F} = R_{C \leftarrow I} R_{I \leftarrow G} =: R_{C \leftarrow G}$
- Clone IMU 위치: $\frac{\partial \mathbf{X}_C}{\partial \mathbf{p}_{I_i,G}} = -R_{C \leftarrow G}$
- Clone IMU 회전 (perturbation $R_{IG} \rightarrow R_{IG} \exp([\delta \boldsymbol{\theta}]_{\times})$):

$$\frac{\partial \mathbf{X}_C}{\partial \delta \boldsymbol{\theta}_{IG}} = -R_{C \leftarrow I} \cdot [R_{I \leftarrow G}(\mathbf{p}_F - \mathbf{p}_{I_i,G})]_{\times}$$

(부호 $\ominus$, skew 가 바깥. 이전 본 가이드의 부호/위치 부정확분을 정정.)

- Camera-imu extrinsic ($R_{C \leftarrow I}, \mathbf{p}_{I,C}$): 본 코드는 calibration off 라 고정 — H 에 이 column 비활성 (OV 코드의 if-branch).

본 코드는 UpdaterHelper::get_feature_jacobian_full 에서 이 모든 항을 계산. FEJ on 일 때 위 식의 $R_{IG}, \mathbf{p}_{I_i,G}, \mathbf{p}_F$ 가 first estimate 로 평가.

4.3 Null-space projection (Givens)

각 feature 에서:

$$H_x * \delta x + H_f * \delta p_F + \text{noise} = r \quad (\text{residual})$$

$H_f \in \mathbb{R}^{2m \times 3}$ 의 left null-space basis $A^T \in \mathbb{R}^{(2m-3) \times 2m}$ 를 구하면:

$$A^T H_f = 0 \Rightarrow A^T r = A^T H_x \delta x + A^T \text{noise}$$

→ feature state 사라짐. EKF measurement 차원 $2m - 3$.

Givens rotation 으로 구현: H_f 를 upper triangular 로 만드는 직교 회전 들을 누적. 그 회전들의 맨 아래 $2m - 3$ 행 이 A^T .

본 코드: `UpdaterHelper::nullspace_project_inplace` — in-place QR-via-Givens.

4.4 Chi² gate

각 feature 의 nullspace-projected residual r' ($2m - 3$ 차원) 와 covariance:

$$S = H'_x P H'^T_x + \sigma_{px}^2 I_{2m-3}$$

Mahalanobis distance:

$$\chi^2 = r'^T S^{-1} r'$$

Gate: $\chi^2 > \alpha \cdot \chi_{\text{table}}^2[2m - 3]$ 이면 drop.

- $\alpha = \text{chi2_multiplier}$ (본 코드 5.0, OV header default; OV yaml 은 1.0)
- $\chi_{\text{table}}^2[k] = \chi^2$ 분포의 95% quantile, dof k . 본 코드에서 ctor 가 미리 1~500 까지 채워둠.

튜닝 영향:

- $\alpha \uparrow \rightarrow$ gate 느슨 $\rightarrow$ outlier 통과 가능성 $\uparrow$
- $\alpha \downarrow \rightarrow$ gate 엄격 $\rightarrow$ 정상 inlier 도 reject 가능

4.5 Measurement compression (QR)

N 개 feature 의 projected H'_x, r' 를 모두 쌓으면 $H_{\text{big}} \in \mathbb{R}^{M \times d_{\text{state}}}$ (M 매우 큼).

QR 분해: $H_{\text{big}} = Q[T_H; 0]$ where $T_H \in \mathbb{R}^{d_{\text{state}} \times d_{\text{state}}}$ upper triangular.

$$Q^T r_{\text{big}} = \begin{bmatrix} r_T \\ r_0 \end{bmatrix}, \quad H \delta x \approx r_T \implies \text{EKF with } T_H, r_T$$

→ EKF update 차원이 d_{state} 로 줄어듦. 계산량 $O(M^3) \rightarrow O(M d_{\text{state}}^2)$.

본 코드: `UpdaterHelper::measurement_compress_inplace`.

4.6 T_rectcam1_imu 유도 (cycle 5)

Given:

- T_rectcam0_imu 알려져 있음 (cycle 4 에서 계산)
- baseline $b > 0$ (rectified cam0 frame 에서 cam1 origin 의 x 좌표)

Rectified frame 에서 cam1 의 원점은 cam0 frame 에서 $(+b, 0, 0)$:

$$T_{rc0 \leftarrow rc1} = \begin{bmatrix} I & (+b, 0, 0)^T \\ 0 & 1 \end{bmatrix}$$

Inverse:

$$T_{rc1 \leftarrow rc0} = \begin{bmatrix} I & (-b, 0, 0)^T \\ 0 & 1 \end{bmatrix}$$

Compose:

$$T_{rc1 \leftarrow I} = T_{rc1 \leftarrow rc0} \cdot T_{rc0 \leftarrow I}$$

→ 회전 부분은 동일 ($T_{rc1 \leftarrow rc0}$ 의 $R = I$), translation 만 $(-b, 0, 0)$ 더해짐.

KITTI 0117 검증:

T_rectcam0_imu.t = $(-0.314, \dots, \dots)$

baseline = 0.537

T_rectcam1_imu.t.x = $-0.314 + (-0.537) = -0.851$ ✓

Part 5 — 디버그 사이클 1~5

본 part 는 방법론의 학습 가치 가 가장 높음. 왜 그 가설을 세웠고 어디서 틀렸는지의 흐름.

5.1 사이클별 한 줄 요약

사이클	핵심 가설	결과
1	G1 gravity 부호 / G2 quat round-trip / G3 frame convention	G1✗ G2✓ G3 의심
2	gram_schmidt 포트로 R_GtoI 재구축	yaw flip → hybrid → 악화
3	H1 motion contamination / H2 χ^2 lock-out / H3 noise tuning	H1✗ (reasoning trap) H2✓ H3~
4	H1 reframing (등속) + 5 fixes	full 2,821 cm (mono 한계)
5	Stereo backend 재구축	full 256 cm (LC 1.7× 까지)

5.2 Cycle 1 — 첫 발산의 출발

현상: cycle 1 init (ba=0, bg=mean_gyro) 으로 KITTI 0117 660-frame run. ATE = 53,279 cm.

G1: gravity sign 부호 의심

- 가설: OpenVINS Global z 가 down 일 수 있다 (cycle 1 의 잘못된 추측). gravity_mag 를 -9.81 로 줘 보자.
- 결과: ✗. ATE 1,332,060 cm 으로 악화. gravity_mag = +9.81 옳음.

G2: R1 quat round-trip 의심

- 가설: Hamilton R0_wi → JPL q_GtoI → back-to-matrix 의 round-trip 에서 오류.
- 검증: $|\text{state_} \rightarrow _imu \rightarrow \text{Rot}() - R_GtoI|$ 측정.
- 결과: ✓. **1.13e-16** (numerical zero). round-trip 정확. 가설 부정.

G3: frame convention mismatch

- 가설: world z = up 인데 OV Global z 가 down 이면 gravity cancellation 이 wrong sign 으로.
- 정황: stationary v.z 가 매 frame 0.05 m/s 씩 누적. drift 패턴.
- 결과: cycle 2 에서 검증.

5.3 Cycle 2 — gram_schmidt 포트 + 진실 발견

시도 1: `ov_init::gram_schmidt(z_axis = a_avg / |a_avg|)` 로 R_GtoI 빌드 (Init from accel only).

- 결과: 50f ATE 579 cm (개선!) 그러나 $R_{wi_diag} = (-0.996, -0.999, +0.995)$ — yaw 180° flip.
- 발견: **OpenVINS Global z = up** (cycle 1 G1 의 추측이 반대).
- 문제: gram_schmidt 가 $e_2 \times z$ 로 임의 yaw 선택. KITTI 차량의 +x forward 와 반대.

시도 2 (hybrid): $R_GtoI = R0_wi.T$ (LC 의 yaw 유지) + $ba = a_avg - R_GtoI * g_inG$ (OV 식 ba).

- 결과: yaw align 회복 ✓, v.z residual 감소 (0.148 → 0.044). 그러나 full ATE 1,599,060 cm 으로 악화.
- 진단: $ba = (0.023, 0.009, +0.257)$. z 성분 +0.26 m/s² 가 propagator 의 과보정 원인.
- 추정 (당시): KITTI 첫 1초가 진짜 정지 가 아니라 motion contamination 으로 ba 오염. → 잘못된 진단 (cycle 4 에서 반박됨).

5.4 Cycle 3 — 방법론 첫 적용 (그리고 H1 의 reasoning trap)

사이클 3 부터 [[feedback-problem-first-then-opensource]] 적용: Step1 정의 → Step2 opensource read → Step3 가설별 검증, 각 단계 별도 commit.

Step 1 — Problem statement (no code)

문제 정의 commit (06d026b): 현상 / 측정값 / 가설 H1~H3 / 성공 기준 / 분할 commit plan 을 journal 에만 기록.

Step 2 — Opensource read (no code)

64233b0 commit: OpenVINS 의 StaticInitializer, UpdaterMSCKF, NoiseManager 를 전체 컨텍스트 로 읽음.

- 발견: OV StaticInitializer 가 two sliding window + init_imu_thresh gate 사용 (jerk 검출).
- 발견: KAIST yaml (driving) 의 noise/threshold 값: init_imu_thresh = 0.5, sigma_a = 5.886e-3, sigma_pix = 1.5.

Step 3 — 가설 검증

H1: KITTI 0117 motion contamination

- tools/probe_kitti_imu_variance.cpp 신규 (~140줄, standalone).
- 측정: 첫 5초 의 sliding 0.5s window 의 a_var.
- 결과: 70.8% windows < KAIST gate 0.5. 대부분 정지로 보임.
- 결론 (당시): H1 **x 부정**. motion contamination 아님.

여기서 **reasoning trap**. a_var 가 낮음을 "정지" 로 해석. 사실은 "가속도 변화 없음". 등속 운동도 a_var ≈ 0 . 이 함정이 cycle 4 에서 반박됨.

H2: χ^2 gate lock-out

- 변경 (6357e42): ba=0 으로 revert + accept/drop counter.
- 결과: 50f cumul accept = **8.4%**. χ^2 gate 가 거의 모든 measurement reject.
- 결론: H2 **✓ 적중**. lock-out 메커니즘 확인.
- 메커니즘: init covariance 작음 $\rightarrow$ P 작음 $\rightarrow \chi^2$ S 작음 $\rightarrow$ 작은 residual 도 큰 $\chi^2 \rightarrow$ reject $\rightarrow$ state 보정 안 됨 $\rightarrow$ P 못 grow $\rightarrow$ 다음도 reject. 악순환.

H3: KAIST yaml noise

- 변경 (6c3d4a6): sigma_a 2.0e-3 $\rightarrow$ 5.886e-3, sigma_pix 1.0 $\rightarrow$ 1.5.
 - 결과: accept 8.4% $\rightarrow$ 33.3%. ATE 940 $\rightarrow$ 956 cm (50f, 동등). full 53k $\rightarrow$ 57k cm.
 - 결론: H3 **~ 부분**. accept 향상 확인. 그러나 ATE 미개선.
 - 새 패턴: full run frame 0~250 accept 70% 가다가 250+ 재차 lock-out.
- $\rightarrow$ cycle 3 commit + tag v0.3.1-cycle3-locked-out. 방법론 첫 마일스톤.

5.5 Cycle 4 — Reframing + 5 fixes

협업 AI 가 진단. 본인 (Claude) 의 H1 reasoning trap 을 발견.

H1 의 진짜 의미 (재해석)

- KITTI 0117 의 GT 가 첫 5초에 35.5 m 이동. 평균 ~ 7 m/s ≈ 25 km/h.
- 즉 시작부터 등속 주행 중.
- 등속 = a_var ≈ 0 . cycle 3 의 probe_kitti_imu_variance 가 "stationary" 라고 오인.
- 진짜 원인: $v_0 = 0$ init $\rightarrow$ propagator 가 가속 없는 운동 으로 예측 $\rightarrow$ scale collapse.

Applied 5 fixes (모두 함께 작동해야 효과)

1. **H5 init covariance** — OV StaticInitializer 식의 init cov 명시 적용
2. **Rectified cam0 extrinsic** — $T_{rectcam0_imu} = R_{rectcam0_cam0} * T_{cam0_imu}$ (전 cycle 까진 raw cam0 사용)
3. **Stereo v0 seed** (★ 결정적) — 첫 stereo frontend 변위를 IMU velocity 초기화
4. **max_clone_size** 11 $\rightarrow$ 30
5. **sigma_pix** 1.5 $\rightarrow$ 5.0 (+ env override MSCKF_*)

Results

- 50f ATE: 67.88 cm (LC 50f 103 보다 좋음)
- Full 660f: 2,821.51 cm (cycle 1 의 19× 개선)
- 그러나 mono backend 의 한계 — full LC parity 미달 (152 vs 2821, 18× gap)

Architectural lesson 발견 ([[feedback-match-baseline-architecture]])

- KITTI/EuRoC 가 stereo 데이터
- LC EKF 가 stereo update
- 그런데 MSCKF 는 암묵적으로 mono backend 로 결정 (kCamId=0, num_cameras=1)
- → 18× gap 의 본질 은 센서 정보량 차이 (알고리즘 차이가 아님)
- cycle 5 에서 stereo 로 재구축 결정

5.6 Cycle 5 — Stereo TC MSCKF

3 buildable commits

3α (3630dec) — StereoTracker cam1 픽셀 노출:

- prev_pts_r_ + stereo_valid_ 멤버 추가
- tracked_points_right(), stereo_valid() accessor
- bootstrap + subsequent + new feature 의 right 픽셀 보존
- LC 영향 없음

3β (19195c2) — MsckfPipeline + run_vio:

- ctor 에 T_cam1_imu 추가
- num_cameras = 2, calib + intrinsic + CamRadtan 등록
- feed_camera(t, left, right) split. cam0 = alive 정의, cam1 = subset
- T_rectcam1_imu = T_rectcam1_rectcam0 * T_rectcam0_imu 계산

3γ — first run:

- 50f ATE 22.33 cm (mono 3×, LC 5× 우위)
- Full 660f 256.26 cm (mono 11×, LC 1.7× gap)
- cumul accept 86.6%. frame 250+ lock-out 패턴 사라짐.

→ tag v0.4.0-stereo-msckf. architectural fidelity 회복 마일스톤.

5.7 방법론 누적 학습

사이클	패턴	교훈
1~2	가설 → 즉시 코드 → 롤백	비효율. sunk cost ↑
3	Step1/2/3 분리 + 별도 commit	시행착오가 git 에 보존
4	협업 AI 의 외부 시각	echo chamber 방지
5	architectural lesson 발견	baseline 의 sensor 구성 매칭 필수

→ [[feedback-problem-first-then-opensource]], [[feedback-match-baseline-architecture]],
[[feedback-a-var-is-not-stationarity]] 의 세 메모리 항목.

Part 6 — 결과와 LC 갭 분석

6.1 전체 결과 표

사이클	Backend	Config	50f ATE	Full 660f ATE	accept_pct
baseline	LC EKF	stereo VO + EKF	—	152.96 cm	—
cycle 1	mono	default	940 cm	53,279 cm	—
cycle 2 시도 1	mono	gram_schmidt	579 cm	—	—
cycle 2 시도 2	mono	hybrid	990 cm	1,599,060 cm	—
cycle 3 H3b	mono	KAIST noise	956 cm	57,861 cm	27.8%
cycle 4 final	mono	5 fixes	67.88 cm	2,821.51 cm	69.7%
cycle 5	stereo	full	22.33 cm	256.26 cm	86.6%

6.2 성공기준 평가

기준	결과
50f < 500 cm	✓ 22 cm (22× margin)
Full < 5000 cm	✓ 256 cm
50f LC parity (< 200)	✓ 5× 우위 (LC 50f 103 cm)
Full LC parity (< 200)	! 1.7× gap (LC 152 vs MSCKF 256)
Stretch full < 100	✗

6.3 왜 full 에서 LC 가 더 좋은가 (1.7× gap 의 원인)

핵심 단서: **50f** 는 **MSCKF** 가 5× 우위, **full** 은 **LC** 가 1.7× 우위. 즉 시간이 갈수록 MSCKF 의 drift 가 더 커진다.

원인 후보 (우선순위 순):

1순위 — Mono 튜닝의 carry-over

cycle 4 의 mono 튜닝값이 stereo 에 그대로:

파라미터	현재	Mono 에서 잡은 이유
sigma_pix = 5.0	mono chi ² lock-out 회피용	1 feature × 1 cam = 측정 부족 → gate
chi2_multipler = 5.0	OV header default	(해당)
max_clone_size = 30	window 크게 → 더 많은 feature 살림	mono 정보 부족 보완

→ 즉시 검증 가능. **Part 7** 의 튜닝 가이드 참조.

2순위 — stereo_valid_ false 비율 미계측

stereo_klt 가 자주 실패하면 cam1 measurement 누락 → 사실상 partial mono.

- bootstrap/new feature: ORB descriptor match (정밀)

- surviving feature: stereo_klt (gradient, drift 가능)

instrumentation 안 함. 만약 false 비율 30%+ 이면 cycle 5 의 stereo 가 간헐적 으로만 작동.

3순위 — stereo_klt 의 sub-pixel drift

KLT 는 gradient descent. 텍스처 약한 곳, occlusion, repetitive pattern 에서 sub-pixel drift 가능. cam0 의 KLT 는 PnP RANSAC 으로 filter 되지만 cam1 의 KLT 는 filter 안 됨. → 시간 누적 시 systematic bias.

4순위 — 아키텍처 차이 (LC 의 강점)

- LC: stereo_tracker 내부 PnP RANSAC → outlier 강하게 reject → 깨끗한 pose 만 EKF 에 제공
 - MSCKF: raw feature 받아 χ^2 로 reject — PnP RANSAC 만큼 뽀뽀하지 않을 수 있음
- 원리상 MSCKF 가 더 좋아야 하나 튜닝 으로 못 따라잡을 때 발생.

정리

LC 가 본질적으로 더 좋아서가 아니라:

- 현재 튜닝이 mono 시대의 잔재
- stereo 매칭 품질 미측정

→ 1순위 가설 (sigma_pix + chi2_multipler retune) 이 맞으면 한 줄 변경 으로 LC 넘을 가능성 있음.

Part 7 — 튜닝 가이드

본 part 의 모든 파라미터는 영향 메커니즘 + 어디서 변경하나 + 어떤 측정값으로 평가 의 3개 축으로 설명.

7.1 sigma_pix — 픽셀 측정 노이즈

무엇

각 feature 의 픽셀 위치 측정 오차의 표준편차 (px).

어디서

src/msckf_pipeline/msckf_pipeline.cpp:MsckfPipeline::MsckfPipeline:

```
upd_opts->sigma_pix    = env_double("MSCKF_SIGMA_PIX", 5.0);
upd_opts->sigma_pix_sq = pow(upd_opts->sigma_pix, 2);
```

메커니즘

1. EKF update 의 measurement covariance $R = \sigma_{px}^2 I$
2. χ^2 gate 의 $S = H_x P H_x^T + \sigma_{px}^2 I$
3. EKF Kalman gain $K \propto S^{-1}$

값 변경 시 영향

sigma_pix	효과
↑ (예: 5.0 → 10.0)	measurement 신뢰도 ↓ → update 가 적게 보정 → 더 propagator 신뢰
↓ (예: 5.0 → 1.5)	measurement 신뢰도 ↑ → update 가 많이 보정 → χ^2 gate 더 엄격 (S 작음)

Reference 값

Dataset	OV yaml up_msckf_sigma_px
EuRoC MAV	1.0
KAIST (driving)	1.5
TUM VI	1.0
RealSense	1.0~1.5

→ KITTI 도 **1.5 권장** (KAIST 와 sensor 유사).

측정 방법

```
for v in 1.0 1.5 2.0 3.0 5.0; do
    MSCKF_SIGMA_PIX=$v ./build/run_vio config/_full_msckf.yaml --engine msckf 2>&1 \
        | grep "ATE RMSE" | sed "s/^/sigma_pix=$v /"
done
```

목표: full ATE 가 최소화되는 sweet spot 찾기.

7.2 chi2_multipler — χ^2 gate 강도

무엇

χ^2 gate 의 threshold 배율. $\chi_{\text{check}}^2 = \alpha \cdot \chi_{\text{table}}^2[\text{dof}]$.

어디서

같은 위치:

```
upd_opts_ -> chi2_multipler = env_double("MSCKF_CHI2_MULT", 5.0);
```

메커니즘

- $\alpha = 1.0 \rightarrow$ 정확히 95% confidence (정상 inlier 의 5% reject)
- $\alpha = 5.0 \rightarrow$ 매우 느슨 (이론적으로 ~99.5% 통과)

값 변경 시 영향

α	효과
5.0 (현재)	매우 permissive. outlier 통과 가능성 $\uparrow$
1.0 (OV yaml)	95% confidence. outlier 강하게 reject
0.5	매우 엄격. inlier 도 일부 reject

Reference

OV header default = 5.0, **OV yaml 은 모든 dataset 에서 1.0.** header value 는 사실상 사용 안 됨.

$\rightarrow$ KITTI 도 **1.0 권장.**

1순위 추천 sweep

```
for c in 1.0 2.0 3.0 5.0; do
    MSCKF_CHI2_MULT=$c MSCKF_SIGMA_PIX=1.5 \
        ./build/run_vio config/_full_msckf.yaml --engine msckf 2>&1 \
        | grep -E "ATE RMSE|accept_pct" | tail -3 \
        | sed "s/^/chi2=$c sigma=1.5  /"
done
```

목표: full ATE 와 cumul accept_pct 의 trade-off 관찰. accept_pct 가 너무 떨어지면 (~30% 미만) 다시 lock-out.

7.3 NoiseManager — IMU 노이즈 4종

무엇

IMU process noise. propagator covariance 의 Q 를 결정.

어디서

src/msckf_pipeline/msckf_pipeline.cpp:

```
NoiseManager noises;
noises.sigma_a = env_double("MSCKF_SIGMA_A", 5.886e-3); // 가속도 noise density
noises.sigma_a_2 = pow(noises.sigma_a, 2);
// 미노출: sigma_w, sigma_wb, sigma_ab (defaults 사용)
```

include/msckf/utils/NoiseManager.hpp 의 default:

파라미터	의미	Default	KAIST yaml
sigma_w	gyro white noise (rad/s/√Hz)	1.7e-4	1.7e-4
sigma_a	accel white noise (m/s ² /√Hz)	2.0e-3	5.886e-3
sigma_wb	gyro bias random walk (rad/s ² /√Hz)	1.9e-5	1.0e-5
sigma_ab	accel bias random walk (m/s ³ /√Hz)	3.0e-3	1.0e-4

메커니즘

propagator 의 Q block 들:

- $Q_{ww} = \sigma_w^2/dt$
- $Q_{aa} = \sigma_a^2/dt$
- $Q_{wbwb} = \sigma_{wb}^2 \cdot dt$
- $Q_{abab} = \sigma_{ab}^2 \cdot dt$

→ propagation 시 state covariance 가 얼마나 grow 할지 결정.

왜 $/dt$ vs $\cdot dt$: 두 종류의 noise 가 단위가 다름.

- White noise (gyro/accel 측정 잡음): power spectral density (PSD) σ^2 in (unit²/Hz). 연속시간 $\int_0^{dt} \mathbf{n}(\tau) d\tau$ 의 분산이 $\sigma^2 \cdot dt$. 그러나 IMU 의 적분된 측정값 으로 보면 분산이 σ^2/dt 의 형태로 covariance 에 들어감 (sample rate $1/dt$ 의 Riemann sum).
- Random walk (bias drift): bias 자체가 random walk 면 시간 Δt 후의 분산이 $\sigma^2 \cdot \Delta t$. 직관적.

이 관계는 IMU spec sheet 의 단위에서도 명시:

- sigma_w 단위: rad/s/√Hz → 시간 적분 시 $\sqrt{dt}$ 로 scaling
- sigma_wb 단위: rad/s²/√Hz → 시간 적분 시 $\sqrt{dt}$ 로 scaling

KAIST IMU spec 의 단위가 이 형태 → yaml 의 값을 그대로 sigma_w/sigma_wb 에 넣음.

값 변경 시 영향

sigma_a	효과
↑	propagator 의 P grow rate ↑ → update 의 영향력 ↑ → measurement 신뢰 ↑
↓	propagator 더 신뢰 → update 가 적게 보정

cycle 3 H3a 에서 sigma_a 2.0e-3 → 5.886e-3 변경의 의미: propagator 의 V 와 bias growth 가 빨라지므로 update 가 영향력 있는 state correction 가능.

Reference

본 코드는 sigma_a 만 변경. 다른 3개는 default 유지. **추가 sweep 후보:**

- sigma_ab 3.0e-3 → 1.0e-4 (KAIST 값) — accel bias 의 RW 가 덜 자유롭게 변하게

측정

bias 추적 plot 추가 필요. 현재는 cur_v 만 출력. 추가 instrumentation 작업 후 sweep 가능.

7.4 max_clone_size — Sliding window 크기

무엇

sliding window 에 몇 개 의 camera clone 유지할지.

어디서

```
opts.max_clone_size = env_int("MSCKF_MAX_CLONES", 30);
```

메커니즘

- 큰 window: 한 feature 가 더 많은 frame 에서 보일 수 있음. multi-state constraint 의 정보량 ↑. 그러나 계산 비용 도 ↑.
- 작은 window: 빠르지만 feature 의 일찍 dropping → 정보 손실.

KITTI 10 Hz: 30 clones = 3초 window.

값 변경 시 영향

max_clone	효과
11 (cycle 3)	mono 부족 → 정보 부족 더 부족
30 (현재)	mono 보완 OK. stereo 에서는 오래된 outlier 누적 위험
20	절충
11 으로 다시 줄여보기	stereo + 짧은 window 가 lock-out 없이 작동하는지

Reference

OV yaml 들의 max_clone_size: 일반적으로 11 (driving) ~ 15 (drone). **stereo 에서는 11 이 정석.**

Sweep

```
for n in 11 15 20 30; do
    MSCKF_MAX_CLONES=$n ./build/run_vio config/_full_msckf.yaml --engine msckf 2>&1 \
        | grep "ATE RMSE" | sed "s/^/clones=$n /"
done
```

7.5 Init covariance — set_initial_covariance 의 5개 block

무엇

State 의 초기 공분산 — q, p, v, bg, ba 각 3-DOF block.

어디서

```
init_cov.block<3,3>(0,0) = std::pow(0.02, 2) * I_3; // q (0.02 rad std  $\approx 1.1^\circ$ )
init_cov.block<3,3>(3,3) = std::pow(0.05, 2) * I_3; // p (5 cm std)
init_cov.block<3,3>(6,6) = std::pow(0.01, 2) * I_3; // v (1 cm/s std)
// bg/ba: base  $(0.02)^2 (= 4e-4)$ 
```

메커니즘

너무 작으면: state 가 과신 됨. update 가 거의 영향 못 줌 (chi² lock-out 의 시초). 너무 크면: 초기 transient 가 길어짐. 첫 몇 frame 의 sensitivity ↑.

Reference

OV StaticInitializer 의 값을 그대로 사용. 잘 검증됨. 바꿀 필요 적음.

변경 후보 (cycle 6 영역)

- v block 의 0.01 m/s std 가 작을 수 있음. cycle 4 의 v0 seed 값 (~6 m/s) 의 불확실성 을 반영하려면 더 큰 값 (예: 0.5 m/s) 필요.

7.6 v₀ seed — 초기 IMU velocity

무엇

KITTI 0117 의 등속 시작 처리 위한 velocity 초기화.

어디서 (실제 수식)

apps/run_vio.cpp 의 stereo 첫 displacement → IMU world velocity:

1. StereoTracker 의 첫 frame pose: $T_{W \leftarrow C_0^{(0)}} = (R_{WC_0}, t_{WC_0})$ (LC 와 anchor 공유)
2. 현재 frame pose: $T_{W \leftarrow C_0^{(t)}}$
3. 두 시점의 IMU world 위치:

$$\mathbf{p}_{I,W}^{(t)} = T_{W \leftarrow C_0^{(t)}} \cdot T_{C_0 \leftarrow I} \cdot \mathbf{0} = R_{WC_0^{(t)}} t_{C_0 I} + t_{WC_0^{(t)}}$$

여기서 $T_{C_0 \leftarrow I}$ = cam0-imu extrinsic (raw, rectified 아님).

4. Velocity:

$$\mathbf{v}_0 = \frac{\mathbf{p}_{I,W}^{(t)} - \mathbf{p}_{I,W}^{(0)}}{t - t_0}$$

5. Sanity check: $0.1 < |\mathbf{v}_0| < 50$ m/s.
 6. msckf->seed_initial_velocity(v0) 호출 (한 번만).
- 이 식이 frontend pose (cam0 in world) 를 IMU world velocity 로 변환하는 전체 chain.

메커니즘

stationary init 가정이 깨졌을 때 (등속) propagator 가 처음에 잘못된 v 로 시작하면 매 frame 누적 drift. v_0 seed 로 회복.

Reference

cycle 4 의 결정적 fix. KITTI 0117 에서 $|v_0| \approx 6.2$ m/s 측정됨.

변경 후보

- v_0 seed 후 의 init covariance v block 도 같이 키우기 (예: 0.5 m/s std). 현재 0.01 m/s std 는 6.2 m/s seed 와 부정합.
- 두 frame 이 아닌 더 많은 frame 의 평균 으로 v_0 측정.

7.7 FEJ on/off

무엇

First Estimates Jacobian — 자코비안을 매번 최신 추정값에서 갱신할지, 최초 추정값 에서 고정 할지.

어디서

```
opts.do_fej = true;
```

메커니즘

- FEJ on: yaw / position drift 의 일관성 보존. unobservable 방향이 정확히 unobservable 로 유지.
- FEJ off: 자코비안 정확. 그러나 spurious information 누적 (관측 못 한 방향에도 약간의 정보 누적 → 잘못된 신뢰).

Reference

OV 의 모든 yaml 이 default on. KITTI 도 on 권장.

변경 후보

cycle 6 의 진단 차원에서 잠깐 off 해보고 ATE 변화 확인. 차이 크면 FEJ 의 영향이 큼.

7.8 feat_rep_msckf — Feature representation

무엇

Feature 의 3D 위치 표현 방식.

어디서

```
opts.feat_rep_msckf = LandmarkRepresentation::Representation::GLOBAL_3D;
```

옵션과 수학적 의미

옵션	Feature state	parameterize
GLOBAL_3D (현재)	$\mathbf{p}_F = (X, Y, Z)_G$	global frame 의 Cartesian
ANCHORED_3D	$\mathbf{p}_{F,A} = (X, Y, Z)_A$	anchor cam frame 의 Cartesian
ANCHORED_MSCKF_INVERSE_DEPTH	(α, β, ρ) where $\alpha = X/Z, \beta = Y/Z, \rho = 1/Z$	anchor cam frame normalized
ANCHORED_INVERSE_DEPTH_SINGLE	$\rho = 1/Z$ only	anchor cam frame (u_n)

여기서 anchor = feature 의 첫 measurement 가 발생한 cam clone 의 frame. 모든 후속 frame 의 measurement 는 anchor 와의 상대 pose 로 해석.

메커니즘 — 왜 inverse depth 가 outdoor 에 유리

GLOBAL_3D 의 문제: 깊이 Z 가 클수록 (예: 100m) 픽셀 1 px 변화가 ΔZ 로 큰 값. 자코비안 $\partial u / \partial Z = -f_x X / Z^2$ 이 $1/Z^2$ 로 감소 $\rightarrow$ 작은 픽셀 noise 도 Z 의 큰 uncertainty 로 amplify $\rightarrow$ covariance 가 비선형적으로 분포 $\rightarrow$ linearization 부정확.

Inverse depth 의 해결: $\rho = 1/Z$ 로 parameterize 하면 자코비안 $\partial u / \partial \rho = -f_x X$ 가 깊이 무관 상수 $\rightarrow$ linearization 이 큰 깊이에서도 잘 정의. 또한 $\rho \in [0, \rho_{\max}]$ 의 bounded 영역.

KITTI 의 특성: 도로/건물의 큰 깊이 (50-100m) 가 많음. ground feature 는 작은 깊이 (3-10m). depth 분포가 광범위 $\rightarrow$ linearization 의 영향이 큼. inverse depth 가 원리상 더 적합.

현재 본 코드의 한계: GLOBAL_3D 사용. cycle 5 에서 anchor_cam_id = 0 으로 defensive set 하지만 anchor representation 안 씀. 큰 깊이 feature 의 update 가 수치적으로 불안정 할 가능성.

$\rightarrow$ Cycle 6 후보: feat_rep_msckf = ANCHORED_MSCKF_INVERSE_DEPTH 시도.

Reference

KITTI outdoor (맨 빌딩, 도로) 에는 inverse depth 가 원리상 더 적합. 그러나 OV 의 default 는 dataset 마다 다름.

변경 후보

cycle 6 에서 ANCHORED_MSCKF_INVERSE_DEPTH 시도. anchor_cam_id 명시 필요 (현재 0 으로 세팅됨).

7.9 Stereo_klt 의 stereo_valid 비율

무엇

매 frame 의 후속 feature 중 stereo KLT 가 성공 한 비율.

어디서

src/frontend/stereo_tracker.cpp 의 process() 내부:

```
auto pts_r_curr = stereo_klt(gray_l, gray_r, tracked_pts, stereo_ok);
for (size_t i = 0; i < tracked_pts.size(); ++i) {
    const bool ok = stereo_ok[i];
    tracked_valid.push_back(ok);
}
```

메커니즘

stereo_valid_[i] == false 이면 그 feature 는 cam1 측정값 없음 → mono measurement 만 update 에 기여. 너무 많이 false 이면 사실상 mono.

측정 방법 (계측 추가 필요)

```
// StereoTracker 에 추가:
int stereo_ok_count = 0;
for (auto v : tracked_valid) if (v) ++stereo_ok_count;
std::cerr << "[stereo_valid] " << stereo_ok_count << "/" << tracked_valid.size()
            << " (" << 100.0 * stereo_ok_count / tracked_valid.size() << "%)\n";
```

이 계측을 cycle 6 의 시작 commit 으로 추가 후, full run 의 평균 비율 측정.

영향

- 비율 > 80%: stereo update 정상 작동
- 비율 50~80%: 절반 정도 partial mono. ATE 에 영향 중간
- 비율 < 50%: 사실상 mono 에 가까움. 갭의 주요 원인 가능

7.10 우선순위 종합 (cycle 6 진입 시)

순위	변경	비용	기대 효과
1	MSCKF_SIGMA_PIX=1.5 + MSCKF_CHI2_MULT=1.0 (env, 코드 0줄)	분 단위	full ATE 1/2~1/3
2	max_clone_size=11 또는 15 sweep	분	full ATE 추가 1/2
3	stereo_valid 비율 계측 추가	~30분	진단만
4	비율이 낮으면 stereo_klt 의 search range / 품질 개선	~2시간	full ATE 영향 큼
5	feat_rep_msckf = ANCHORED_MSCKF_INVERSE_DEPTH 시도	~1시간	outdoor 큰 깊이 stab
6	bg/ba random walk 튜닝 (sigma_ab 1.0e-4 KAIST 값)	env	bias trajectory smoot

→ 1~3 까지만 해도 LC parity (152 cm) 가능성 매우 높음.

Part 8 — 방법론 회고

8.1 메모리에 박힌 4개 교훈

본 프로젝트에서 재발 방지용 으로 메모리에 저장된 항목:

feedback-problem-first-then-opensource

- 코드 수정 전에 (1) 문제 정의 (2) opensource 참고 (3) 가설별 검증을 별도 commit 으로
- cycle 1~2 의 즉시 적용 → 롤백 패턴이 비용 컸음
- cycle 3 부터 적용. 시행착오가 git history 에 보존되어 학습 가능 한 형태로 누적.

feedback-a-var-is-not-stationarity

- IMU accel variance 가 낮음 $\neq$ stationary
- 등속 운동 (KITTI 0117) 은 $a_{var} \approx 0$
- 정지 판정에는 비-IMU 신호 (camera disparity / GT) 동시 필요
- cycle 3 H1 의 reasoning trap 박제

feedback-match-baseline-architecture

- baseline 의 sensor 구성을 명시적으로 매칭
- KITTI/EuRoC stereo 인데 MSCKF 를 암묵 mono 로 결정 → cycle 1~4 sunk cost
- "simplicity 로 빠르게" 같은 암묵 trade-off 금지

feedback-navisys-commit-structure

- feat + docs 의 commit pair, debug → fix → docs 패턴
- 마일스톤마다 tag + CHANGELOG entry
- 시행착오도 commit 으로 보존 → 학습 자산

8.2 작업 자체의 패턴 회고

패턴 1: 가설을 물질화 (verbalize) 한 시점이 늦었다

cycle 1~2 는 머리속 가설 을 바로 코드 로 변환. 가설이 글로 쓰여지기 전 이라 비교 가능 하지 않았음. cycle 3 부터 journal 의 problem statement 라는 외부 기록 도입.

패턴 2: opensource 의 전체 컨텍스트 읽기 vs 발췌 복사

cycle 2 의 gram_schmidt 단편 발췌 → yaw flip. cycle 3 부터 전체 흐름 읽고 맥락 이해 후 적용.

패턴 3: 진단 도구의 재사용 가능성

tools/probe_kitti_imu_variance.cpp 는 cycle 3 의 산물이지만 영구 보존. 향후 다른 KITTI sequence / EuRoC 분석에도 재사용. → 디버그 도구는 그 자체로 자산.

패턴 4: 외부 시각의 가치

cycle 4 의 협업 AI 가 본인 (Claude) 의 H1 reasoning trap 발견. 같은 모델의 echo chamber 가 위험. 정기적 외부 시각 도입 가치.

Part 9 — Quick reference

9.1 빌드

```
# 환경: WSL Ubuntu 의 cmake
cd /mnt/d/02_research/04_cpp_seg_msckf_vio
cmake --build build -j4 --target run_vio
```

9.2 실행

```
# 50f (빠른 검증)
./build/run_vio config/kitti_raw_2011_09_26_drive_0117_local_50f.yaml --engine msckf

# Full 660f
./build/run_vio config/_full_msckf.yaml --engine msckf

# LC 비교
./build/run_vio config/_full_lc.yaml --engine lc

# IMU variance probe (cycle 3 H1)
./build/probe_kitti_imu data/kitti_raw/2011_09_26/2011_09_26_drive_0117_sync 5.0 0.5
```

9.3 Env override (튜닝용)

```
MSCKF_SIGMA_PIX=1.5    ./build/run_vio config/_full_msckf.yaml --engine msckf
MSCKF_CHI2_MULT=1.0    ./build/run_vio ... --engine msckf
MSCKF_MAX_CLONES=11    ./build/run_vio ... --engine msckf
MSCKF_SIGMA_A=5.886e-3 ./build/run_vio ... --engine msckf
```

9.4 핵심 파일 위치

위치	내용
src/msckf_pipeline/msckf_pipeline.cpp	어댑터 본체 (~330줄)
include/msckf_pipeline/msckf_pipeline.hpp	어댑터 인터페이스
apps/run_vio.cpp	엔트리 + LC/MSCKF 분기 + v0 seed
src/frontend/stereo_tracker.cpp	프론트엔드 + cycle 5 cam1 노출
include/msckf/state/Propagator.hpp/cpp	OV propagator
include/msckf/update/UpdaterMSCKF.hpp/cpp	OV updater
include/msckf/feat/Feature.hpp	Feature 의 cam_id keyed map
tools/probe_kitti_imu_variance.cpp	cycle 3 H1 도구
docs/insight/20260522_tc_msckf_port_journal.md	전체 cycle 1~5 evidence

9.5 Git tag history

Tag	마일스톤
v0.1.0-lc-ekf	LC EKF baseline (retroactive)
v0.2.0-msckf-port-p2	OV types/utils/cam/state 포트 완료
v0.3.0-msckf-runs	KITTI crash-free (P5)
v0.3.1-cycle3-locked-out	방법론 첫 적용 (cycle 3)
v0.3.2-cycle4-mono-final	mono 라인 종결 (50f 68, full 2821 cm)
v0.4.0-stereo-msckf	stereo 작동 (50f 22, full 256 cm)

9.6 ATE 목표

기준	현재	목표 (cycle 6)
50f	22 cm	< 50 cm (이미 달성)
Full	256 cm	< 152 cm (LC parity)
Stretch full	256 cm	< 100 cm

부록 A — 본 가이드의 관련 메모리·문서

- 메모리 (C:\Users\taehwan.lee\.claude\projects\D--02-research-04-cpp-seg-msckf-vio\memory\):
 - project_tc_msckf_port.md — 현재 메인 작업
 - feedback_problem_first_then_opensource.md
 - feedback_a_var_is_not_stationarity.md
 - feedback_match_baseline_architecture.md
 - feedback_navisys_commit_structure.md
 - feedback_doc_structure.md
- 본 repo:
 - docs/insight/20260522_tc_msckf_port_journal.md — cycle 별 evidence
 - docs/Practice/tc_msckf_port_progress.md — 진척 추적

- docs/insight/20260520_vio_lvio_msckf_thinking_cheatsheet.md — VIO/LVIO 전 반 사고방식
- CHANGELOG.md — 마일스톤 narrative

부록 B — 다음 세션 진입 트리거

트리거	작업
cycle 6 시작	LC parity 추구. sigma_pix/chi2 sweep 부터.
학습 Q&A	본 가이드의 어느 section 에 대해 질문.
튜닝 sweep	Part 7 의 특정 파라미터 sweep 실행.

끝.